

Architecture Document: AI Task Processing Platform

Prepared by: S Sabareesh

1. Executive Summary

This document outlines the architecture for a robust, scalable AI Task Processing Platform. The system leverages an asynchronous, event-driven design to handle high-volume text processing tasks, utilizing Kubernetes for orchestration and Argo CD for automated GitOps-based deployment.

2. System Architecture & Workflow

The platform employs a decoupled architecture:

- **Frontend:** A React-based interface enables users to register, log in, and submit text processing tasks.
- **Backend API:** Node.js + Express manages authentication (JWT) and queues task metadata in Redis.
- **Worker Service:** A Python-based worker service processes operations (uppercase, lowercase, reversal, word count).
- **Database:** MongoDB stores user profiles, task status, and results.

3. Scaling Strategy

Horizontal Autoscaling (HPA): To manage 100,000 tasks/day, the system implements a Kubernetes HPA that monitors the worker deployment's CPU utilization and Redis queue depth.

Throughput: Averaging ~1.2 tasks/second, the architecture handles bursts of 10x volume by decoupling the API and Worker via Redis, preventing bottlenecks.

4. Database & Reliability

Indexing Strategy: The MongoDB *tasks* collection utilizes compound indexes on *userId* and *createdAt* to ensure efficient retrieval of task history.

Redis Failure Handling: Redis is configured with **Append Only File (AOF)** persistence. This ensures that in the event of a container restart, the worker recovers the queue state, preventing task loss.

5. DevOps & Deployment

CI/CD Pipeline: GitHub Actions automates the lifecycle: linting, multi-stage Docker builds, and automated image pushing.

GitOps (Argo CD): Kubernetes manifests are stored in an infrastructure repository. Argo CD monitors this, maintaining parity between git and the cluster.

Environments: Staging and production isolation are enforced via dedicated Kubernetes namespaces.

6. Security Architecture

The platform ensures data integrity through:

- **Auth:** Stateless JWT authentication with bcrypt-hashed passwords.
- **Defense:** Helmet middleware for security headers and rate limiting to mitigate DoS risks.